ADVANCED PYTHON PROGRAMMING
LAB ASSESSMENT – 2

P. Ganesh Prakadesh

23MIC0020

QUESTION:

32. Longest Valid Parentheses

Hard | Topics | Companies

Given a string containing just the characters `'('` and `')'`, return *the length of the longest valid (well-formed) parentheses substring*.

Example 1:

```
Input: s = "(()"
Output: 2
Explanation: The longest valid parentheses substring is "()".
```

Example 2:

```
Input: s = ")()())"
Output: 4
Explanation: The longest valid parentheses substring is "()()".
```

Example 3:

```
Input: s = ""
Output: 0
```

INPUT:

String of parantheses

EXPECTED OUTPUT:

Integer denoting the length of the longest valid parantheses substring of the input

SOLUTION:

</> Code

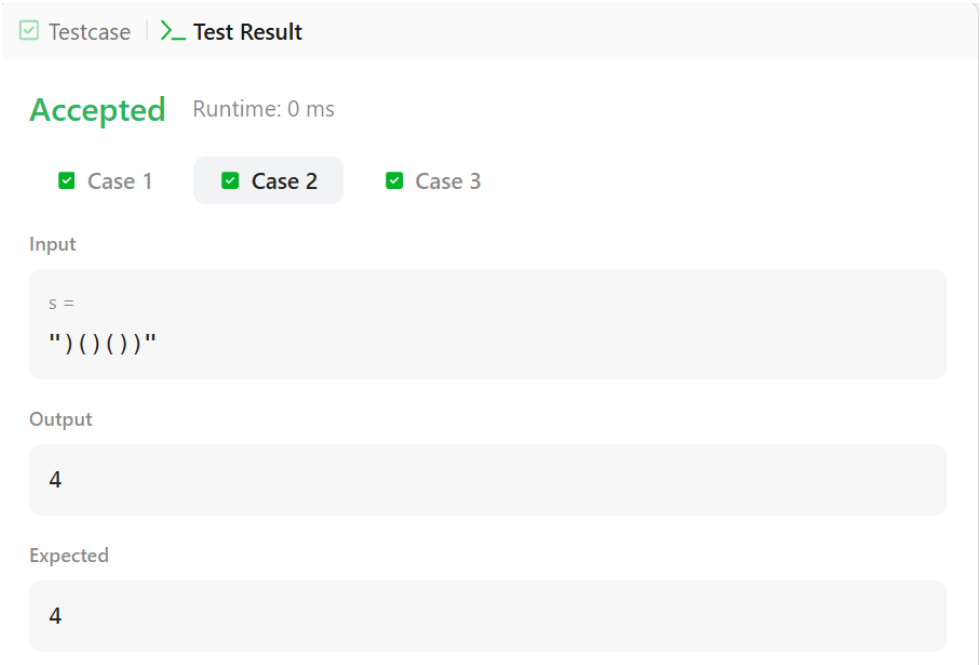
Python3 Auto

```python
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        stack = [-1]
        max_len = 0

        for i in range(len(s)):
            if s[i] == "(":
                stack.append(i)
            else:
                stack.pop()
                if len(stack) == 0:
                    stack.append(i)
                else:
                    max_len = max(max_len, i - stack[-1])

        return max_len
```

TEST CASES:



DATA STRUCTURE USED:

A stack is used here since it follows LIFO and can use pop() to track valid parantheses. The invalid ones' index are stored in the stack. It makes it easy to find the length of the valid parantheses.

ERROR DOCUMENTATION:

Runtime Error

```
IndexError: pop from empty list
    ~~~~~~~~~^^
    stack.pop()
Line 10 in longestValidParentheses (Solution.py)
    ret = Solution().longestValidParentheses(param_1)
Line 41 in _driver (Solution.py)
    ~~~~~~~^^
    _driver()
Line 56 in <module> (Solution.py)
```

Ran into a runtime error when initialising the stack as empty instead of having a -1 in it. Pop would then run on an empty stack, deeming the code invalid.

QUESTION:

189. Rotate Array

Medium | Topics | Companies | Hint

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

INPUT:

A list of integers and an integer k to represent the number of steps the list has to be rotated

EXPECTED OUTPUT:

The resultant list after rotating the input list k times

SOLUTION:

Code

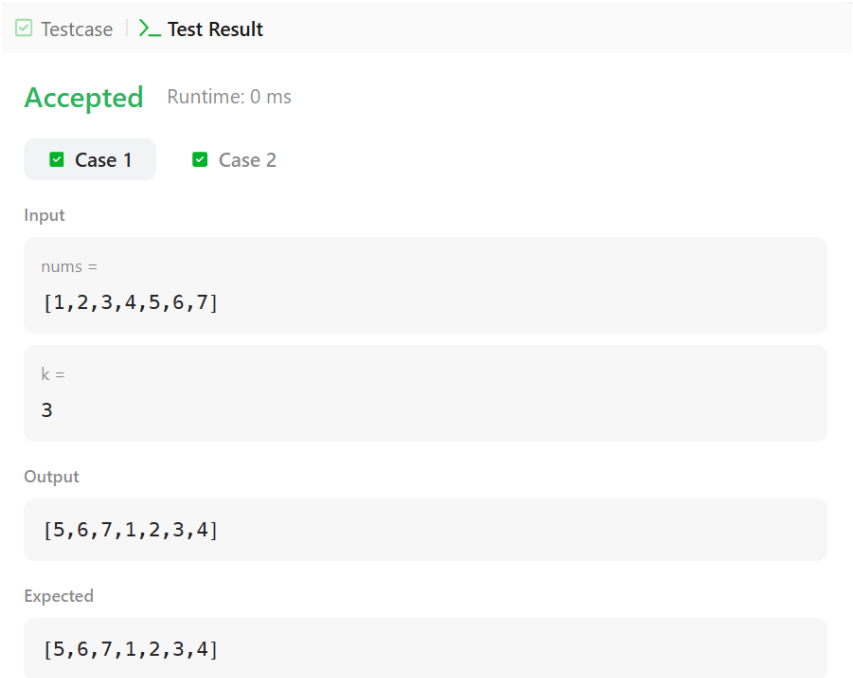
Python3 Auto

```python
class Solution:
    def rever(self, nums, start, end):
        while start < end:
            nums[start], nums[end] = nums[end], nums[start]
            start += 1
            end -= 1

    def rotate(self, nums: list[int], k: int) -> None:
        length = len(nums)
        k %= length
        if k == 0:
            return

        someMid = length - k
        self.rever(nums, 0, someMid - 1)
        self.rever(nums, someMid, length - 1)
        self.rever(nums, 0, length - 1)
```

TEST CASES:



DATA STRUCTURE USED:

For this question, we use the indexes to rotate the elements and rearrange the array. So a list is used. This helps in easy element access and assignment.

QUESTION:

204. Count Primes

Medium | Topics | Companies | Hint

Given an integer `n`, return *the number of prime numbers that are strictly less than* `n`.

Example 1:

```
Input: n = 10
Output: 4
Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.
```

Example 2:

```
Input: n = 0
Output: 0
```

Example 3:

```
Input: n = 1
Output: 0
```

INPUT:

Integer n till which prime numbers are checked

EXPECTED OUTPUT:

An integer denoting the number of prime numbers less than n

SOLUTION:

Code

Python3 Auto

```python
class Solution:
    def countPrimes(self, n: int) -> int:
        count = 0  # Stores number of primes

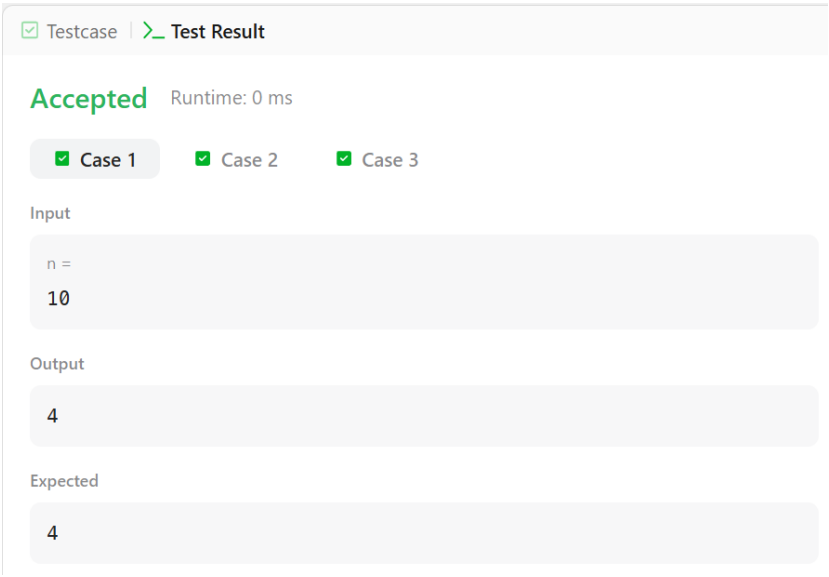
        # Check every number from 2 to n-1
        for i in range(2, n):
            prime = True

            # Check if i has any factors
            for j in range(2, int(i ** 0.5) + 1):
                if i % j == 0:
                    prime = False
                    break

            # If no factors were found, it is prime
            if prime:
                count += 1

        return count
```

TEST CASES:



DATA STRUCTURE USED:

No data structure is utilised in this code. The code simply uses variables to assign and reassign values as the count is increased, based on the number of prime numbers encountered while doing so.

ERROR DOCUMENTATION:

Incorrect output obtained because of starting the for loop from 1 instead of 2

QUESTION:

215. Kth Largest Element in an Array

Medium | Topics | Companies

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array*.

Note that it is the kth largest element in the sorted order, not the kth distinct element.

Can you solve it without sorting?

Example 1:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

INPUT:

A list of integers as input and an integer k to denote the kth largest element to be printed

EXPECTED OUTPUT:

An integer which is the kth largest element in the input list

SOLUTION:

Code

Python3 Auto

```python
class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        a = nums.copy()  # Copy the original list

        # Find the largest element k times
        for i in range(k):
            largest = a[0]

            # Find largest element
            for x in a:
                if x > largest:
                    largest = x

            # kth largest found
            if i == k - 1:
                return largest

            # Remove only one occurrence
            a.remove(largest)
```

TEST CASES:

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =

[3,2,1,5,6,4]

k =

2

Output

5

Expected

5

DATA STRUCTURE USED:

A list is used to store the numbers. It allows us to access, search, and remove elements easily. A copy of the list is created so the original list is not modified.

ERROR DOCUMENTATION:

Forgot to take a copy of the original list which ended up removing all occurrences of the largest element in the list

---

QUESTION:

442. Find All Duplicates in an Array

Medium | Topics | Companies

Given an integer array `nums` of length `n` where all the integers of `nums` are in the range `[1, n]` and each integer appears **at most twice**, return *an array of all the integers that appears **twice***.

You must write an algorithm that runs in `O(n)` time and uses only *constant* auxiliary space, excluding the space needed to store the output

Example 1:

```
Input: nums = [4,3,2,7,8,2,3,1]
Output: [2,3]
```

Example 2:

```
Input: nums = [1,1,2]
Output: [1]
```

Example 3:

```
Input: nums = [1]
Output: []
```

INPUT:

A list of integers

EXPECTED OUTPUT:

A list of integers containing the duplicate elements found in the input list

SOLUTION:

Code

Python3 Auto

```python
class Solution:
    def findDuplicates(self, nums: List[int]) -> List[int]:
        result = []
        n = len(nums)
        for i in range(n):
            num = abs(nums[i])
            idx = num - 1
            if nums[idx] < 0:
                result.append(num)
            nums[idx] *= -1
        return result
```

TEST CASES:

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =

[4,3,2,7,8,2,3,1]

Output

[2,3]

Expected

[2,3]

DATA STRUCTURE USED:

A list is used here. The input itself is a list so the elements are marked using indices and duplicates appended into a new resultant list using append()

ERROR DOCUMENTATION:

Used idx = num which gave a runtime error of “list index out of range”. It was changed to num-1 to fix the error

Runtime Error

```
IndexError: list index out of range
      ~~~~^^^^^
    if nums[idx] < 0:
Line 8 in findDuplicates (Solution.py)
    ret = Solution().findDuplicates(param_1)
Line 36 in _driver (Solution.py)
    ~~~~~~~^^
    _driver()
Line 51 in <module> (Solution.py)
```